Demo Paper Title

1st Given Name Surname
Department of Informatics (or whatever your department is)
Universität Hamburg
Hamburg, Germany
email address or ORCID

2nd Given Name Surname
Department of Informatics (or whatever your department is)
Universität Hamburg
Hamburg, Germany
email address or ORCID

3rd Given Name Surname
Department of Informatics (or whatever your department is)
Universität Hamburg
Hamburg, Germany
email address or ORCID

***Abstract*—Small retailers such as cafés must repeatedly decide how much of each ingredient to purchase, balancing the risk of running out of popular items against the cost of over-ordering perishable stock. We present an interactive demo that forecasts daily ingredient demand for a café by combining structured per-cup sales records from PostgreSQL with semi-structured recipe data from MongoDB in a multi-database pipeline. Drink-level sales are forecast independently using additive Triple Exponential Smoothing and mapped to ingredient quantities through the recipe database, aggregating over all drinks that share an ingredient. Through a graphical interface, visitors can manually tune the model's smoothing parameters or let a grid search select them automatically, then inspect the resulting forecasts for individual ingredients across both a held-out evaluation period and the days beyond the available data. Beyond producing plausible purchasing forecasts, the demo lets visitors directly compare ingredients that draw on many drinks against ones that depend on only one or two, exposing firsthand how this difference affects forecast stability, and more generally where a common forecasting method remains reliable and where its assumptions break down.**

***Index Terms*—time series forecasting, Holt-Winters, exponential smoothing, demand forecasting, multi-database systems, PostgreSQL, MongoDB, interactive visualization**

I. Introduction

Cafés and other small retailers face the challenge of how much of each ingredient should be ordered for the coming days or weeks. Ordering too little risks running out of popular products and disappointing customers while ordering too much leads to waste and higher cost. Accurate demand forecasting directly addresses this problem by estimating future ingredient consumption from historical sales patterns [1].

Beyond the retail context, forecasting time-series data arises in a wide range of applications, including short-to-medium-term predictions of business metrics like sales, revenue and demand planning [2]. The coffee sales scenario used in this demo serves as an example of the broader family of exponential smoothing methods [3]. The project demonstrates how structured sales records and semi-structured recipe data can be combined in a multi-database architecture to result in a forecast of daily ingredient requirements. The system introduces a graphical user interface through which users can select an ingredient and adjust key model parameters like the three smoothing parameters α, β, γ and immediately observe their effect on the forecast.

II. System/Solution/Architecture Overview

Figure 1 gives an overview of the system architecture developed for this demo. The pipeline follows a multi-database design that reflects the heterogeneous nature of the underlying data: structured, transactional sales records on one side, and semi-structured, schema-flexible recipe data on the other.

Historical per-cup sales transactions are stored in a **PostgreSQL** relational database, following a normalized schema. Ingredient recipes for each drink, which ingredients are required and in what quantity, are stored as documents in a hosted **MongoDB Atlas** cluster, since recipes are naturally nested and variable in structure (differing ingredient counts, units, and preparation steps per drink) and do not benefit from a fixed relational schema.

The forecasting core is implemented in **Java 17**. On startup, a `DataLoader` component queries PostgreSQL for daily per-drink cup counts, filling any days without recorded sales with zero to obtain a contiguous daily time series per drink. Each series is then forecast independently using additive Triple Exponential Smoothing (Section II-A), producing both an evaluation forecast over a held-out period and a forecast for the days beyond the available data. An `IngredientMapper` component subsequently multiplies each drink's forecasted daily cup count by the corresponding ingredient quantities from the MongoDB recipes and aggregates the results across all drinks that share an ingredient (e.g., milk is used in lattes, cortados, and hot chocolate alike). The resulting ingredient-level forecasts, together with the actual historical amounts, are written back into an `ingredient_forecast` table in PostgreSQL, keyed by date and ingredient.

The graphical interface is implemented in **Python 3** using `PyQt5` for the UI and `matplotlib` for plotting, with `psycopg2` providing direct read access to the `ingredient_forecast` table. This separation allows the

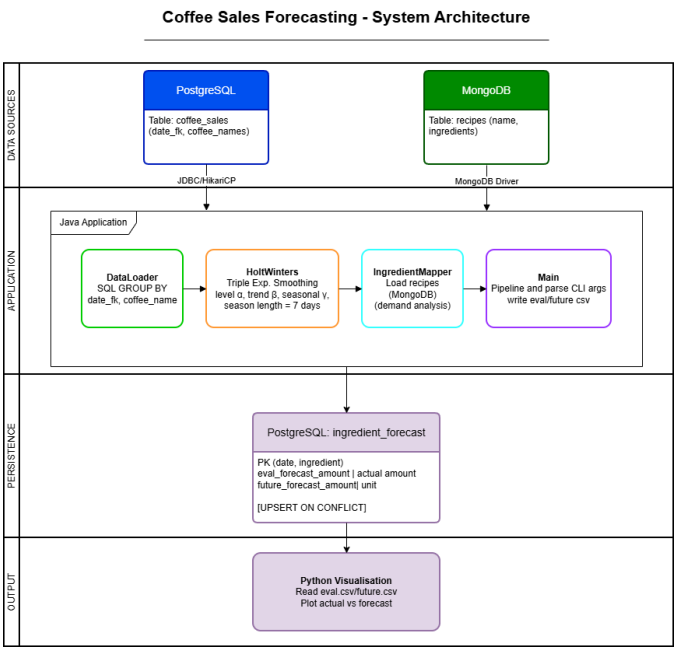

Fig. 1. Multi-database architecture: PostgreSQL stores structured sales data, MongoDB stores semi-structured recipes, the Java forecasting core connects both, and the Python/PyQt5 GUI visualizes the results.

interface to remain purely presentational: all forecasting and ingredient-mapping logic resides in the Java backend, while the GUI is responsible only for triggering a (re-)run of the Java forecasting jar with user-chosen parameters and for querying and rendering the persisted results. A grid search over $\alpha, \beta, \gamma \in \{0.1, \ldots, 0.9\}$, implemented in Python for rapid iteration, is offered as an alternative to manual parameter tuning; it re-runs the Holt-Winters computation in-memory for each parameter combination and each drink, selecting the combination that minimizes the root-mean-squared error against the held-out evaluation period.

A. Forecasting Approach

Ingredient demand is derived from per-drink sales forecasts computed using **additive Triple Exponential Smoothing (Holt-Winters method)**, which decomposes each daily time series into a level, a trend, and a weekly (period $L = 7$) seasonal component:

$$
\begin{aligned}
l_t &= \alpha(y_t - s_{t-L}) + (1-\alpha)(l_{t-1} + b_{t-1}),\\
b_t &= \beta(l_t - l_{t-1}) + (1-\beta)b_{t-1},\\
s_t &= \gamma(y_t - l_t) + (1-\gamma)s_{t-L},
\end{aligned}
$$

The h-step-ahead forecast is then given by $\hat{y}_{n+h} = l_n + hb_n + s_{n+h-L\lceil h/L \rceil}$. Negative forecasts, which can occur for low-volume drinks, are clipped to zero since cup counts cannot be negative. Level and trend are initialized from the mean over the first season and zero, respectively, and the seasonal components are initialized as the deviation of each of the first L observations from that mean.

This method was chosen over simpler alternatives (e.g., a single- or double-exponential-smoothing model, or a naive moving average) because coffee sales exhibit a clear and stable weekly cycle. Weekday versus weekend demand differs substantially for most drinks in the dataset, in addition to a slower-moving trend over the observed year. Triple Exponential Smoothing captures both effects simultaneously with only three tunable parameters and closed-form update equations, making it well suited to a demo where users are meant to interactively adjust α, β, and γ and immediately observe the effect on the forecast, and where a grid search over the parameter space must remain computationally cheap. Its short-to-medium-term forecasting behavior is well documented in the forecasting literature [1]–[3], and it is widely used as a standard baseline for seasonal demand forecasting in retail and inventory settings, which matches the ingredient-ordering use case targeted by this demo.

III. Demonstration Proposal/Scenarios/Workflow

This section describes how users can experience our forecasting pipeline. Users can fine-tune the model manually or find suitable parameters automatically via a grid search, and see where the model behaves well as well as where its assumptions start to break down.

A. Setup

The demo runs on a laptop with an Apple Silicon M3 CPU and 16GB of RAM on macOS, however, the demo can also be run on less powerful hardware, given the moderate size of the dataset. The forecasting core is implemented in Java 17 and built with Maven into a self-contained jar. Historical sales are stored in PostgreSQL 16, while the ingredient recipes used to translate drink forecasts into ingredient quantities are kept in a hosted MongoDB Atlas cluster. Parameter search, visualization, and the graphical interface are implemented in Python 3, using `psycopg2` for database access, `pandas` for data handling, and `matplotlib` together with `PyQt5` for plotting and the GUI itself. A full grid search over the smoothing parameters α, β, γ at a step size of 0.1 evaluates $9^3 = 729$ parameter combinations per drink and completes within a few seconds on this hardware.

B. User Experience

Users can interact with the demo through a graphical interface (see Fig. 2). First, the user can select the parameters manually or run a grid search to find the best parameters for the forecasting model. Afterwards, the user can run the forecasting model and visualize the results for one ingredient. For the selected ingredient, two plots are created, one comparing actual sales against the forecast for the held-out evaluation period, and one showing the forecast for the upcoming days beyond the available data. After the forecast is generated and without running the forecasting model again, the user can select another ingredient in a drop-down menu and visualize the results for that ingredient. Visitors are also encouraged to compare ingredients that differ in how many drinks contribute

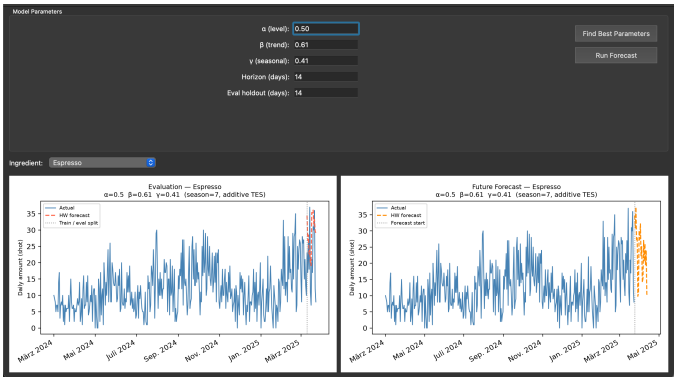

Fig. 2. The graphical interface showing model parameters, the ingredient selector, and the resulting evaluation and future forecast plots.

to them. Ingredients drawn from several drinks tend to produce a stable forecast, since noise in any single drink's prediction is averaged out, whereas ingredients tied to only one or two drinks can inherit that drink's forecasting errors directly. This makes it possible to observe, within the same interface, both cases where the model performs convincingly and cases where its assumptions no longer hold.

C. Conclusion

The demo shows a complete path from raw, per-cup sales records to ingredient-level forecasts that could plausibly support purchasing decisions in a small coffee shop. Its particular value lies not in presenting a single polished result, but in letting visitors interactively expose the conditions under which a common forecasting method remains reliable and the conditions under which it does not, using the same pipeline and the same data throughout.

References

[1] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 2nd ed. OTexts, Melbourne, Australia, 2018. [Online]. Available: https://otexts.com/fpp2/

[2] C. C. Holt, "Forecasting seasonals and trends by exponentially weighted moving averages," *International Journal of Forecasting*, vol. 20, no. 1, 2004.

[3] E. S. Gardner, "Exponential smoothing: The state of the art," *Journal of Forecasting*, vol. 4, no. 1, pp. 1–28, 1985.